GitHub Checkout: Old vs New Version — Complete Comparison

Table of Contents

1. Overview
2. Background: Why a New `checkout` Was Needed
3. Installation & Availability
4. Core Syntax Comparison
5. Switching Branches
6. Creating Branches
7. Restoring Files
8. Detached HEAD State
9. Tracking Remote Branches
10. Stash Interactions
11. Sparse Checkout
12. Orphan Branches
13. Error Messages & User Experience
14. Safety & Accidental Data Loss
15. Performance
16. Scripting & Automation
17. Interoperability
18. Full Feature Matrix
19. When to Use Which Command
20. Migration Cheat Sheet

Overview

Git has traditionally used a single command — `git checkout` — to handle many different, often unrelated tasks: switching branches, creating branches, restoring files, detaching HEAD, and more. This overloaded design caused confusion, especially for beginners.

Starting with **Git 2.23 (August 2019)**, Git introduced two new commands to split and replace the responsibilities of `git checkout`:

New Command	Replaces
`git switch`	Branch switching and creation operations
`git restore`	File restoration and unstaging operations

Both commands were initially marked **experimental** and became **stable** in **Git 2.28 (July 2020)**.

> **Key point:** `git checkout` is **not removed** and remains fully functional. The new commands are additive alternatives, not replacements that break existing workflows.

Background

Why `git checkout` Was Considered Problematic

`git checkout` suffered from **command overloading** — the same command performed completely different actions depending on context:

```bash
git checkout main            # Switches branch
git checkout -b feature      # Creates AND switches to a new branch
git checkout -- file.txt     # Discards working directory changes to a file
git checkout HEAD~2          # Detaches HEAD to a specific commit
git checkout v1.0            # Detaches HEAD to a tag
git checkout main -- file.txt  # Copies a file from another branch (no branch switch!)
```

These operations are semantically unrelated, and the ambiguity caused real problems:

- `git checkout file.txt` with no `--` separator would attempt a branch switch if `file.txt` happened to match a branch name.
- New users frequently ran `git checkout file.txt` intending to restore a file, not realizing it would **silently discard all uncommitted changes** with no confirmation.
- Scripts were fragile because the behavior was context-dependent.

Installation & Availability

Aspect	`git checkout` (Old)	`git switch` / `git restore` (New)
Available since	Git 1.x (original command)	Git 2.23 (August 2019)
Stable since	Always stable	Git 2.28 (July 2020)
Requires installation	No — built into all Git	No — built into Git 2.23+
Works on older Git	Yes	No — requires upgrade
Default in new Git guides	Being phased out in docs	Recommended in official docs

Version check:

```bash
git --version
# Should be 2.23+ to use git switch/restore
```

Core Syntax Comparison

Old: `git checkout`

```bash
git checkout <branch>            # Switch branch
git checkout -b <branch>         # Create and switch
git checkout -B <branch>         # Create/reset and switch
git checkout -- <file>           # Restore file from index
git checkout <commit> -- <file>  # Restore file from commit
git checkout <commit>            # Detach HEAD
git checkout --orphan <branch>   # Create orphan branch
git checkout --track origin/<branch>  # Track remote branch
```

New: `git switch` + `git restore`

```bash
```

```
git switch <branch>              # Switch branch
git switch -c <branch>           # Create and switch
git switch -C <branch>           # Create/reset and switch
git restore <file>               # Restore file from index
git restore --source=<commit> <file>  # Restore file from commit
git switch --detach <commit>     # Detach HEAD explicitly
git switch --orphan <branch>     # Create orphan branch
git switch --track origin/<branch> # Track remote branch (or use -t)
```

Switching Branches

Old: `git checkout`

```bash
# Switch to an existing branch
git checkout main

# Switch to previous branch
git checkout -

# Switch with untracked file handling
git checkout --merge feature-branch
```

New: `git switch`

```bash
# Switch to an existing branch
git switch main

# Switch to previous branch
git switch -

# Switch with merge conflict resolution strategy
git switch --merge feature-branch
# or
git switch -m feature-branch
```

Key Differences

Behavior	`git checkout`	`git switch`
Ambiguous name (branch vs file)	Prefers branch	Always treats as branch
Explicit intent	No	Yes — only switches
`--merge` flag	Supported	Supported
Error on dirty working tree	Sometimes unclear	Clearer error message
`-` shorthand for previous branch	Supported	Supported

Creating Branches

Old: `git checkout`

```bash
# Create and switch to new branch
git checkout -b feature/my-feature

# Create branch from a specific starting point
git checkout -b feature/my-feature origin/main

# Force-create (reset if already exists)
git checkout -B feature/my-feature

# Create local branch tracking a remote branch
git checkout --track origin/feature-x
# or shorthand
git checkout -t origin/feature-x
```

New: `git switch`

```bash
```

```bash
# Create and switch to new branch
git switch -c feature/my-feature

# Create branch from a specific starting point
git switch -c feature/my-feature origin/main

# Force-create (reset if already exists)
git switch -C feature/my-feature

# Create local branch tracking a remote branch
git switch --track origin/feature-x
# or shorthand
git switch -t origin/feature-x

# Auto-create tracking branch (if name matches a remote branch)
git switch feature-x
# Git auto-suggests: "Branch 'feature-x' set up to track remote branch 'feature-x' from 'origin'."
```

Key Differences

Feature	`git checkout`	`git switch`
Flag for create + switch	`-b`	`-c` (more intuitive)
Force create / reset	`-B`	`-C`
Auto-track remote branch by name	Yes	Yes (same behavior)
Semantics of the flag name	`-b` (b = branch)	`-c` (c = create)

Restoring Files

This is where the split is **most significant**. File restoration was folded into `git checkout` historically, but is now a dedicated command.

Old: `git checkout` (File Restoration)

```bash
```

```bash
# Discard working directory changes (restore from index/staging area)
git checkout -- file.txt

# Restore all files in directory
git checkout -- .

# Restore file to its state in a specific commit
git checkout abc1234 -- file.txt

# Restore file from another branch
git checkout main -- src/config.js

# Unstage a file (same as git reset HEAD file)
# (Not directly possible with checkout alone)
```

Danger: The `--` separator is easy to forget, and without it, `git checkout file.txt` will try to switch to a branch named `file.txt`.

New: `git restore`

```bash
# Discard working directory changes (restore from index)
git restore file.txt

# Restore all files
git restore .

# Restore file to its state in a specific commit
git restore --source=abc1234 file.txt
# or using short flag
git restore -s abc1234 file.txt

# Restore file from another branch
git restore --source=main src/config.js

# Unstage a file (move from index back to working tree)
git restore --staged file.txt

# Unstage AND discard working directory changes
git restore --staged --worktree file.txt
# or
git restore -SW file.txt
```

Key Differences

Feature	`git checkout`	`git restore`
Restore from index	`git checkout -- file`	`git restore file`
Restore from commit	`git checkout <sha> -- file`	`git restore --source=<sha> file`
Unstage file	Not directly (use git reset)	`git restore --staged file`
Ambiguity with branch names	Yes — dangerous	No — always file operation
`--` separator required	Yes (for safety)	No — no ambiguity
Target (working tree vs index)	Implicitly working tree	Explicitly controlled via `--staged`, `--worktree`
Restore from another branch	`git checkout branch -- file`	`git restore --source=branch file`

Detached HEAD State

Old: `git checkout`

```bash
# Detach HEAD to a specific commit
git checkout abc1234

# Detach HEAD to a tag
git checkout v2.0.0

# Detach HEAD to a remote branch tip
git checkout origin/main
```

When you run any of these, `git checkout` would silently enter detached HEAD state. For new users, this was confusing because there was no explicit signal that the behavior was different.

New: `git switch`

```bash
```

```
# Explicit detach — must use --detach flag
git switch --detach abc1234

# Detach to a tag
git switch --detach v2.0.0

# Detach to a remote branch tip
git switch --detach origin/main
```

The `--detach` flag is **required** in `git switch` — you cannot accidentally enter detached HEAD state.

Key Differences

Behavior	`git checkout`	`git switch`
Accidental detached HEAD	Possible — implicit	Not possible — must use `--detach`
Warning message in detached HEAD	Sometimes shown	Always shown with clear message
Re-attaching to a branch	`git checkout <branch>`	`git switch <branch>`
Intent clarity	Ambiguous	Explicit

Tracking Remote Branches

Old: `git checkout`

```bash
# Explicit tracking setup
git checkout --track origin/feature-x

# Short form
git checkout -t origin/feature-x

# Auto-tracking (if local name matches remote)
git checkout feature-x
# (Automatically tracks origin/feature-x if it exists)
```

New: `git switch`

```bash
# Explicit tracking setup
git switch --track origin/feature-x

# Short form
git switch -t origin/feature-x

# Auto-tracking (same as before)
git switch feature-x
# (Automatically tracks origin/feature-x if it exists)
```

Both behave identically for tracking. The difference is purely syntactic (`--track` vs `-t`). The auto-track behavior (DWIM — Do What I Mean) is preserved in both.

Stash Interactions

Both old and new commands interact with `git stash` the same way — the stash commands themselves have not changed.

```bash
# Old workflow
git stash
git checkout other-branch
git stash pop

# New workflow
git stash
git switch other-branch
git stash pop
```

However, `git switch` provides slightly clearer error messages when you try to switch branches with uncommitted changes:

Old (`git checkout`) error:

```
error: Your local changes to the following files would be overwritten by checkout:
        file.txt
Please commit your changes or stash them before you switch branches.
```

New (`git switch`) error:

```
error: Your local changes to the following files would be overwritten by checkout:
        file.txt
hint: If you want to keep them, use --merge or stash them.
```

The new command includes a contextual hint, improving the developer experience.

Sparse Checkout

Sparse checkout allows checking out only a subset of files in a large repository.

Old: `git checkout` + Manual Configuration

```bash
# Initialize sparse checkout
git init
git remote add origin <url>
git config core.sparseCheckout true
echo "src/" >> .git/info/sparse-checkout
git checkout main
```

New: `git sparse-checkout` (Introduced in Git 2.25)

While `git switch` is the new branch-switching command, sparse checkout in modern Git is handled by a dedicated subcommand:

```bash
# Initialize and configure sparse checkout
git clone --no-checkout <url>
cd repo
git sparse-checkout init
git sparse-checkout set src/ docs/
git switch main
```

Or with cone mode (faster, directory-based):

```bash
```

```bash
git sparse-checkout init --cone
git sparse-checkout set src docs
git switch main
```

Key Differences

Aspect	Old Approach	New `git sparse-checkout`
Configuration method	Manual edit of `info/sparse-checkout`	Dedicated subcommand
Cone mode (performance)	Not available	`--cone` flag for faster patterns
Listing current patterns	`cat .git/info/sparse-checkout`	`git sparse-checkout list`
Disabling sparse checkout	Manual config reset	`git sparse-checkout disable`
Introduced in	Git 1.x (manual)	Git 2.25 (January 2020)

Orphan Branches

An orphan branch has no commit history — useful for `gh-pages`, documentation, or completely independent project roots.

Old: `git checkout`

```bash
git checkout --orphan gh-pages
git rm -rf .              # Clear working tree
git commit --allow-empty -m "Initial gh-pages commit"
```

New: `git switch`

```bash
git switch --orphan gh-pages
git rm -rf .              # Clear working tree (same step required)
git commit --allow-empty -m "Initial gh-pages commit"
```

The behavior is identical — `--orphan` is supported in both commands.

Error Messages & User Experience

One of the explicit design goals of `git switch` and `git restore` was **better error messages**.

Example 1: Switching with Uncommitted Changes

`git checkout`:

```
error: Your local changes to the following files would be overwritten by checkout:
    README.md
Please commit your changes or stash them before you switch branches.
Aborting
```

`git switch`:

```
error: Your local changes to the following files would be overwritten by checkout:
    README.md
hint: If you want to keep them, use "--merge" or stash them.
hint: To discard your local changes, use "git switch -f"
Aborting
```

The new output tells users **exactly what to do next**.

Example 2: Trying to Restore a File that Doesn't Exist

`git checkout -- nonexistent.txt`:

```
error: pathspec 'nonexistent.txt' did not match any file(s) known to git
```

`git restore nonexistent.txt`:

```
error: pathspec 'nonexistent.txt' did not match any file(s) known to git
```

Similar messages, but `git restore` doesn't have the added complexity of potentially being confused with a branch switch.

Example 3: Attempting to Switch with Untracked Files That Conflict

`git checkout target-branch`:

```
error: The following untracked working tree files would be overwritten by checkout:
    config/local.yaml
Please move or remove them before you switch branches.
Aborting
```

`git switch target-branch`:

```
error: The following untracked working tree files would be overwritten by checkout:
    config/local.yaml
hint: If you want to keep them, commit or stash before switching.
Aborting
```

Safety & Accidental Data Loss

Risk with `git checkout`

The most dangerous aspect of `git checkout` for file operations is that it can **permanently discard uncommitted changes without confirmation**:

```bash
# This SILENTLY destroys all uncommitted changes to file.txt
git checkout -- file.txt
```

There is **no undo** for this. Once run, the changes are gone.

The same silent destruction applies when using:

```bash
git checkout -- .    # Destroys ALL uncommitted changes in the working tree
```

Safety with `git restore`

`git restore` preserves this same behavior (no confirmation prompt) because Git philosophically treats uncommitted changes as the user's responsibility. However, the **explicit and separate nature** of `git restore` makes the intent clearer:

```bash
git restore file.txt    # Still discards changes — but command name signals intent
```

The key safety improvement comes from **separation of concerns**:

- There is no scenario where `git restore file.txt` accidentally switches branches.
- There is no scenario where `git switch branch` accidentally restores files.

Safety Aspect	`git checkout`	`git switch` / `git restore`
Accidental branch switch from file op	Possible	Impossible
Accidental file restore during switch	Possible	Impossible
Accidental detached HEAD	Possible (no flag)	Impossible (requires `--detach`)
Uncommitted change destruction	Silent, no confirmation	Silent, no confirmation (same)
`--force` required for risky ops	Sometimes	`-f`/`--force` explicit option

Performance

For most operations, `git switch` and `git checkout` have virtually identical performance since they rely on the same underlying Git object model and index operations.

Differences may appear in very large repositories:

Scenario	`git checkout`	`git switch`
Switching branches (small repo)	~same	~same
Switching branches (large repo, many files)	Baseline	Slightly optimized in newer Git versions
Sparse checkout operations	Manual/slower	Faster with `--cone` mode (Git 2.25+)
First-time checkout of large repo	Standard	Same

The performance benefit is largely in **sparse checkout** with cone mode, not in the switch/restore commands themselves.

Scripting & Automation

Using `git checkout` in Scripts (Old)

```bash
#!/bin/bash
# Old pattern — checkout overloads make this fragile
BRANCH="feature/deploy"
git checkout "$BRANCH" || git checkout -b "$BRANCH"

# Restore a file — requires -- to prevent branch name ambiguity
git checkout -- config/settings.json

# Extract a file from a specific commit
git checkout "$COMMIT_SHA" -- dist/bundle.js
```

Problems in scripts:

- Return code behavior differs between branch switch and file restore.
- Branch name collisions with file names can cause unexpected behavior.
- The `--` separator must always be remembered.

Using `git switch` + `git restore` in Scripts (New)

```bash
#!/bin/bash
# New pattern — intent is explicit and unambiguous
BRANCH="feature/deploy"
git switch "$BRANCH" 2>/dev/null || git switch -c "$BRANCH"

# Restore a file — no ambiguity possible
git restore config/settings.json

# Extract a file from a specific commit
git restore --source="$COMMIT_SHA" dist/bundle.js
```

Improvements in scripts:

- Return codes are more predictable.
- No need for `--` separator since commands are purpose-specific.
- Easier to read and audit.

> **Note:** For maximum compatibility in scripts targeting diverse Git environments (older servers, CI/CD with older Git), `git checkout` remains safer since it works on all Git versions. Always check `git --version` in your CI/CD pipelines before using `git switch`/`git restore`.

Interoperability

Using Both Commands Together

There is no conflict in using both `git checkout` and `git switch`/`git restore` in the same repository or workflow. They operate on the same underlying Git data structures.

```bash
# Perfectly valid to mix:
git switch main
git checkout -b hotfix/1234      # Old way still works
git restore src/broken.js        # New way for file restore
git checkout -- tests/           # Old way for directory restore
```

IDE and Tool Support

Tool / IDE	`git checkout`	`git switch` / `git restore`
VS Code Source Control	Full support	Full support (2020+)
JetBrains IDEs	Full support	Full support (2021+)
GitHub Desktop	Full support	Full support
GitKraken	Full support	Full support
SourceTree	Full support	Partial (UI may use checkout)
GitHub Actions	Full support	Requires Git 2.23+ on runner
GitLab CI/CD	Full support	Requires Git 2.23+ image

Full Feature Matrix

Feature	`git checkout`	`git switch`	`git restore`
Switch to existing branch	✅	✅	❌
Create and switch to new branch	✅ (`-b`)	✅ (`-c`)	❌
Force create/reset branch	✅ (`-B`)	✅ (`-C`)	❌
Detach HEAD to commit	✅ (implicit)	✅ (`--detach`, explicit)	❌
Create orphan branch	✅ (`--orphan`)	✅ (`--orphan`)	❌
Track remote branch	✅ (`--track`)	✅ (`--track`)	❌
Restore file from index	✅ (`--`)	❌	✅
Restore file from specific commit	✅	❌	✅ (`--source`)
Restore file from another branch	✅	❌	✅ (`--source`)
Unstage file (index → working tree)	❌ (use reset)	❌	✅ (`--staged`)
Restore both staged and working tree	❌	❌	✅ (`-SW`)
Switch to previous branch (`-`)	✅	✅	❌
`-m` / `--merge` on switch	✅	✅	❌
Sparse checkout (manual)	✅	N/A	N/A
Sparse checkout (dedicated subcommand)	N/A	Use `git sparse-checkout`	N/A
Available in Git < 2.23	✅	❌	❌
Stable in Git 2.28+	✅	✅	✅
No branch/file name ambiguity	❌	✅	✅
Explicit intent in command name	❌	✅	✅
Helpful hints in error messages	Partial	✅	✅

When to Use Which Command

Use `git checkout` when:

- Working in environments with Git < 2.23 (legacy servers, old CI images)
- Maintaining scripts that need broad compatibility
- Using features that haven't been replicated exactly (some edge cases)
- Working with teammates who haven't transitioned yet

Use `git switch` when:

- Switching or creating branches in day-to-day development
- You want explicit, readable intent in your command history
- Teaching Git to new developers
- Writing new scripts targeting Git 2.23+

Use `git restore` when:

- Discarding working directory changes to files
- Unstaging files from the index
- Checking out individual files from commits or other branches
- Any time you want to be explicit that you are NOT switching branches

---

Migration Cheat Sheet

Old Command	New Equivalent
`git checkout <branch>`	`git switch <branch>`
`git checkout -`	`git switch -`
`git checkout -b <branch>`	`git switch -c <branch>`
`git checkout -b <branch> <start>`	`git switch -c <branch> <start>`
`git checkout -B <branch>`	`git switch -C <branch>`
`git checkout --track origin/<branch>`	`git switch --track origin/<branch>`

Old Command	New Equivalent
`git checkout --orphan <branch>`	`git switch --orphan <branch>`
`git checkout <sha>`	`git switch --detach <sha>`
`git checkout -- <file>`	`git restore <file>`
`git checkout -- .`	`git restore .`
`git checkout <sha> -- <file>`	`git restore --source=<sha> <file>`
`git checkout <branch> -- <file>`	`git restore --source=<branch> <file>`
`git reset HEAD <file>` *(unstage)*	`git restore --staged <file>`
`git reset HEAD <file>` + `git checkout -- <file>`	`git restore --staged --worktree <file>`

Summary

The introduction of `git switch` and `git restore` represents a deliberate effort by the Git maintainers to address decades of accumulated confusion caused by `git checkout`'s overloaded behavior. The changes can be summarized as:

- **`git switch`** = everything about moving between branches, with safety guardrails
- **`git restore`** = everything about recovering file content, with explicit control over index vs working tree
- **`git checkout`** = still works, still maintained, but no longer the recommended approach for new users or new scripts

The transition is entirely **backward compatible** — there is no urgency to migrate existing workflows. However, for clarity, safety, and alignment with modern Git documentation, adopting `git switch` and `git restore` in new work is strongly recommended.

Reference: Git 2.23 release notes (August 2019), Git 2.28 release notes (July 2020), official Git documentation at https://git-scm.com/docs